

Go for Senior .NET Engineers

Phase 6: Advanced Topics

11 Lessons: Memory Model, GC, Profiling, unsafe, cgo, gRPC, Middleware, Errors, Generics, Reflection, Observability

Current as of Go 1.25.6 — includes Green Tea GC, Flight Recorder, PGO, iter package

Lesson 49: Go Memory Model — Happens-Before & Visibility

Go's memory model defines when reads of a variable in one goroutine are guaranteed to see values written by another goroutine. Coming from C#, think of it as replacing .NET's memory model (volatile, MemoryBarrier, Interlocked) with a simpler set of rules based on synchronization primitives.

49.1 — Core Rule: Happens-Before

```
// A write to variable v in goroutine A is GUARANTEED to be
// visible to a read of v in goroutine B if and only if:
// A's write HAPPENS-BEFORE B's read

// These operations establish happens-before:
// 1. Channel send happens-before the corresponding receive
// 2. Closing a channel happens-before receive of zero value
// 3. mu.Unlock() happens-before the next mu.Lock()
// 4. sync.Once.Do(f) happens-before any Do() returns
// 5. sync.WaitGroup: Add happens-before Go, Done happens-before Wait
```

49.2 — The Danger: No Synchronization

```
// BUG: no happens-before relationship!
var msg string
var done bool

go func() {
    msg = "hello"
    done = true // no synchronization
}()

// This goroutine may NEVER see msg = "hello"
// even if it sees done = true
// The compiler and CPU can reorder these writes
for !done {
    runtime.Gosched()
}
fmt.Println(msg) // might print ""
```

49.3 — C# volatile vs Go Approach

C# / .NET

```
// C# – volatile keyword
private volatile bool _done;
private string _msg;

// volatile prevents reordering
```

Go

```
// Go – use channels or atomics
done := make(chan struct{})
var msg string

go func() {
```

```
// and ensures visibility
_msg = "hello";
_done = true; // volatile write
```

```
// In another thread:
if (_done) // volatile read
    Console.WriteLine(_msg);
```

```
msg = "hello"
close(done) // happens-before
}()
```

```
<-done // synchronizes
fmt.Println(msg) // safe: "hello"
```

TIP: Go has NO volatile keyword. Instead, use channels, sync.Mutex, or sync/atomic for all cross-goroutine communication. If you share memory between goroutines without these primitives, the behavior is undefined.

Lesson 50: Garbage Collector — Tri-Color, GOGC, Green Tea

Go uses a concurrent, tri-color mark-and-sweep garbage collector. Unlike .NET's generational GC with Gen0/Gen1/Gen2 and compaction, Go's GC is non-generational and non-compacting, prioritizing low latency over maximum throughput.

50.1 — .NET GC vs Go GC

Feature	.NET GC vs Go GC
Algorithm	.NET: Generational (Gen0/1/2) Go: Concurrent tri-color mark-sweep
Compaction	.NET: Yes (defragments heap) Go: No compaction
STW pauses	.NET: Varies (ms-seconds) Go: Sub-millisecond
Tuning knob	.NET: Server/Workstation GC Go: GOGC (target overhead %)
Memory limit	.NET: GCSettings Go: GOMEMLIMIT (Go 1.19+)
Generations	.NET: 3 generations Go: None (scans entire heap)

50.2 — Tuning with GOGC and GOMEMLIMIT

```
// GOGC controls GC frequency (default: 100)
// GOGC=100 means: run GC when heap doubles
// GOGC=200 means: run GC when heap triples (less frequent)
// GOGC=50  means: run GC when heap grows 50% (more frequent)
// GOGC=off means: disable GC entirely

GOGC=200 ./myservice    // less GC, more memory usage

// GOMEMLIMIT (Go 1.19+): hard memory limit
// GC becomes more aggressive as limit approaches
GOMEMLIMIT=512MiB ./myservice

// Best practice for containers:
// Set GOMEMLIMIT to ~90% of container memory limit
// GOMEMLIMIT=460MiB for a 512MB container
```

50.3 — Green Tea GC (Go 1.25 Experimental, Default in Go 1.26)

Green Tea is a new GC algorithm that scans memory in contiguous 8 KB spans instead of individual objects. This dramatically improves cache locality and reduces the 35% of CPU cycles previously wasted on memory stalls during marking.

```
// Enable in Go 1.25:  
GOEXPERIMENT=greenteagc go build -o myservice  
  
// Default in Go 1.26 (disable with nogreenteagc)  
// GOEXPERIMENT=nogreenteagc go build  
  
// Results: 10-40% reduction in GC CPU overhead  
// Best improvement: tree/graph structures, JSON parsing,  
//   batch processing of many small objects (<512 bytes)  
// Less improvement: workloads with scattered large objects
```

TIP: For most Go services, the default GOGC=100 works well. Start with GOMEMLIMIT set to 90% of your container's memory. Only tune GOGC if profiling shows GC is a bottleneck. Green Tea GC provides free performance gains on Go 1.26+.

Lesson 51: Profiling — pprof, CPU, Memory, Trace

Go has built-in profiling tools that put BenchmarkDotNet, dotTrace, and dotMemory to shame in terms of ease-of-use. Every Go binary can expose pprof endpoints with a single import.

51.1 — Enabling pprof in Your Service

```
import _ "net/http/pprof" // side-effect import

func main() {
    // Start pprof on a separate port
    go func() {
        log.Println(http.ListenAndServe(":6060", nil))
    }()

    // Your normal server
    http.ListenAndServe(":8080", mux)
}

// Profiles available at:
// http://localhost:6060/debug/pprof/
```

51.2 — Collecting Profiles

```
# CPU profile (30 seconds)
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30

# Heap (memory) profile
go tool pprof http://localhost:6060/debug/pprof/heap

# Goroutine profile (find goroutine leaks)
go tool pprof http://localhost:6060/debug/pprof/goroutine

# Block profile (find lock contention)
go tool pprof http://localhost:6060/debug/pprof/block

# Mutex profile (find mutex contention)
go tool pprof http://localhost:6060/debug/pprof/mutex

# Execution trace (scheduler, GC, goroutine activity)
curl -o trace.out http://localhost:6060/debug/pprof/trace?seconds=5
go tool trace trace.out
```

51.3 — Analyzing with pprof

```
# Interactive mode
go tool pprof cpu.prof
(pprof) top 10      # top 10 hottest functions
(pprof) list myFunc # source-annotated profile
(pprof) web         # open flame graph in browser
```

```
# Direct flame graph
go tool pprof -http=:9090 cpu.prof

# Compare two profiles
go tool pprof -diff_base=old.prof new.prof
```

51.4 — Profile-Guided Optimization (PGO)

```
// Since Go 1.21: use CPU profiles to optimize builds

// 1. Collect a production CPU profile
curl -o default.pgo http://prod:6060/debug/pprof/profile?seconds=60

// 2. Place default.pgo in the main package directory
// 3. Build normally – Go auto-detects and uses it
go build -o myservice

// Typical improvement: 2-7% throughput increase
// Zero code changes required
```

TIP: PGO (Profile-Guided Optimization) is free performance. Collect a CPU profile from production, save it as `default.pgo` in your main package, and rebuild. Go will automatically use it to optimize hot paths. This is the Go equivalent of .NET's tiered compilation, but explicit.

Lesson 52: unsafe — When to Use, When to Avoid

The unsafe package provides operations that step outside Go's type safety, similar to C#'s unsafe keyword and pointer arithmetic. It is used in performance-critical code, interop with C libraries, and some standard library internals.

52.1 — C# unsafe vs Go unsafe

C# / .NET

```
// C# – unsafe block
unsafe {
    int* p = &myInt;
    byte* bp = (byte*)p;
    int size = sizeof(MyStruct);
}

// Requires /unsafe compiler flag
// Used for P/Invoke, perf-critical code
```

Go

```
// Go – unsafe package
import "unsafe"

p := unsafe.Pointer(&myInt)
bp := (*byte)(p)
size := unsafe.Sizeof(MyStruct{})

// No special compiler flag needed
// Used for cgo, perf-critical code
```

52.2 — Common unsafe Operations

```
// Size, alignment, and offset
unsafe.Sizeof(int64(0))    // 8
unsafe.Alignof(int64(0))   // 8
unsafe.Offsetof(s.Field)   // byte offset of field

// Type punning (reinterpret cast)
func float64bits(f float64) uint64 {
    return *(*uint64)(unsafe.Pointer(&f))
}

// String to []byte without copy (common optimization)
func unsafeStringToBytes(s string) []byte {
    return unsafe.Slice(unsafe.StringData(s), len(s))
}
```

WARNING: The unsafe package bypasses Go's memory safety. Code using unsafe can break with any Go version update. It is NOT covered by the Go 1 compatibility guarantee. Only use it when profiling proves it is necessary and there is no safe alternative.

Lesson 53: cgo — Calling C from Go

cgo lets Go call C libraries, analogous to P/Invoke or [DllImport] in C#. However, cgo has significant overhead and complexity costs. The Go community saying is: 'cgo is not Go.'

53.1 — Basic cgo Example

```
package main

/*
#include <stdio.h>
#include <stdlib.h>

void greet(const char* name) {
    printf("Hello from C, %s!\n", name);
}
*/
import "C"
import "unsafe"

func main() {
    name := C.CString("Gopher")
    defer C.free(unsafe.Pointer(name))
    C.greet(name)
}
```

53.2 — P/Invoke vs cgo

C# / .NET

```
// C# - P/Invoke
[DllImport("mylib.so")]
static extern void greet(
    string name);

// Marshalling is automatic
// No manual memory management
// Works with NativeAOT
```

Go

```
// Go - cgo
// #include "mylib.h"
// #cgo LDFLAGS: -lmylib
import "C"

// Manual C string management
// Must free C-allocated memory
// CGO_ENABLED=1 required
```

53.3 — Why Avoid cgo When Possible

Issue	Impact
Build complexity	Requires C compiler, breaks cross-compilation
Performance overhead	~100-200ns per cgo call (goroutine stack switch)
Deployment	Binary is dynamically linked (no FROM scratch)

Debugging	Cannot step through C code in Go debugger
Goroutine safety	C code blocks an OS thread (not a goroutine)

TIP: Before reaching for cgo, look for pure Go alternatives. SQLite? Use modernc.org/sqlite (pure Go). Image processing? Use Go's image packages. Crypto? Go's crypto package is fast and FIPS-certified. cgo should be a last resort.

Lesson 54: gRPC — Protocol Buffers & Service Definitions

gRPC is Google's high-performance RPC framework, and Go is a first-class citizen. It uses Protocol Buffers for serialization (like .NET's protobuf-net) and HTTP/2 for transport. Many Go microservice architectures use gRPC for internal service-to-service communication.

54.1 — Proto File & Code Generation

```
// user.proto
syntax = "proto3";
package user;
option go_package = "github.com/org/svc/pb";

message User {
    string id = 1;
    string name = 2;
    string email = 3;
}

service UserService {
    rpc GetUser(GetUserRequest) returns (User);
    rpc ListUsers(ListUsersRequest) returns (stream User);
}

// Generate: protoc --go_out=. --go-grpc_out=. user.proto
```

54.2 — gRPC Server Implementation

```
type userServer struct {
    pb.UnimplementedUserServiceServer
}

func (s *userServer) GetUser(ctx context.Context, req *pb.GetUserRequest)
(*pb.User, error) {
    user, err := s.repo.FindByID(ctx, req.Id)
    if err != nil {
        return nil, status.Errorf(codes.NotFound, "user %s not found", req.Id)
    }
    return &pb.User{Id: user.ID, Name: user.Name, Email: user.Email}, nil
}

func main() {
    lis, _ := net.Listen("tcp", ":50051")
    s := grpc.NewServer()
    pb.RegisterUserServiceServer(s, &userServer{})
    s.Serve(lis)
}
```

54.3 — ConnectRPC (Modern Alternative)

ConnectRPC (connectrpc.com) is a newer gRPC-compatible framework that works with standard net/http, supports gRPC, gRPC-Web, and a simpler Connect protocol. It is gaining popularity as a lighter-weight alternative to google.golang.org/grpc.

TIP: Use gRPC for internal service-to-service communication where performance matters. Use REST (net/http + JSON) for public APIs and browser clients. ConnectRPC bridges both worlds by supporting gRPC and HTTP/JSON with the same service definition.

Lesson 55: Middleware & HTTP Architecture Patterns

Go's HTTP middleware pattern is based on function composition: a middleware is a function that takes an `http.Handler` and returns a new `http.Handler`. This is simpler than ASP.NET's middleware pipeline but equally powerful.

55.1 — Middleware Chain

```
// Type: func(http.Handler) http.Handler

func Auth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if !isValid(token) {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return // short-circuit – next is NOT called
        }
        // Add user to context
        ctx := context.WithValue(r.Context(), userKey, extractUser(token))
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}

func Recovery(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                slog.Error("panic recovered", "error", err)
                http.Error(w, "internal error", 500)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

// Chain: Recovery(Logger(Auth(mux)))
http.ListenAndServe(":8080", Recovery(Logger(Auth(mux))))
```

55.2 — ASP.NET Middleware vs Go Middleware

C# / .NET

```
// C# – ASP.NET Middleware
app.UseMiddleware<AuthMiddleware>();
app.UseMiddleware<LoggingMiddleware>();

// Or inline:
app.Use(async (ctx, next) => {
    // before
    await next(ctx);
    // after
});
```

Go

```
// Go – function composition
handler := Auth(Logger(mux))

// Or write a chain helper:
func Chain(h http.Handler,
    mws ...Middleware) http.Handler {
    for i := len(mws) - 1; i >= 0; i-- {
        h = mws[i](h)
    }
}
```

```
}  
return h  
}
```

55.3 — Graceful Shutdown

```
srv := &http.Server{Addr: ":8080", Handler: mux}  
  
// Start server in goroutine  
go func() { srv.ListenAndServe() }()  
  
// Wait for interrupt signal  
quit := make(chan os.Signal, 1)  
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)  
<-quit  
  
// Graceful shutdown with timeout  
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)  
defer cancel()  
srv.Shutdown(ctx) // waits for in-flight requests
```

TIP: Always implement graceful shutdown in production services. `http.Server.Shutdown` waits for in-flight requests to complete (up to the context deadline), then closes listeners. This is equivalent to `IHostApplicationLifetime.StopAsync` in .NET.

Lesson 56: Advanced Error Handling — Wrapping, Sentinel, Custom Types

Go 1.13 introduced error wrapping with `fmt.Errorf` and `%w`, plus `errors.Is` and `errors.As` for type-safe error inspection. This is Go's answer to exception hierarchies and inner exceptions in C#.

56.1 — Error Wrapping Chain

```
// Wrap errors to build a chain (like InnerException)
func GetUser(ctx context.Context, id string) (*User, error) {
    row := db.QueryRowContext(ctx, "SELECT ...", id)
    var u User
    if err := row.Scan(&u.ID, &u.Name); err != nil {
        return nil, fmt.Errorf("get user %s: %w", id, err)
        //                                     ^^ wraps original
    }
    return &u, nil
}

// Caller wraps again:
// "handle order: get user abc-123: sql: no rows"
```

56.2 — Sentinel Errors and `errors.Is`

```
// Define sentinel errors (like custom exception types)
var (
    ErrNotFound      = errors.New("not found")
    ErrUnauthorized  = errors.New("unauthorized")
    ErrConflict      = errors.New("conflict")
)

// Check anywhere in the chain (like catch by type)
if errors.Is(err, ErrNotFound) {
    http.Error(w, "not found", http.StatusNotFound)
    return
}

// errors.Is walks the ENTIRE wrap chain
// Similar to checking InnerException recursively in C#
```

56.3 — Custom Error Types and `errors.As`

```
// Custom error with structured data
type ValidationError struct {
    Field  string
    Message string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("validation: %s - %s", e.Field, e.Message)
```

```
}

// Extract typed error from chain
var valErr *ValidationError
if errors.As(err, &valErr) {
    // valErr is populated with the matched error
    fmt.Println(valErr.Field, valErr.Message)
}
```

TIP: `errors.Is` checks for value equality (sentinel errors). `errors.As` checks for type match (custom error structs). Together they replace C#'s `catch(SpecificException ex)` pattern. Always wrap errors with context using `fmt.Errorf("...: %w", err)`.

Lesson 57: Advanced Generics — Constraints, iter, slices, maps

Go 1.21+ introduced the slices, maps, and cmp packages providing generic utility functions. Go 1.23 added iterators (iter package) and range-over-func. These close the gap with C#'s LINQ for collection operations.

57.1 — slices and maps Packages

```
import "slices"
import "maps"

// Sort (in-place)
slices.Sort(names) // like List<T>.Sort()
slices.SortFunc(users, func(a, b User) int {
    return cmp.Compare(a.Age, b.Age)
})

// Search
idx, found := slices.BinarySearch(sorted, target)
slices.Contains(items, "needle")

// Transform
slices.Reverse(items)
slices.Compact(sorted) // remove adjacent duplicates

// Maps
keys := slices.Sorted(maps.Keys(m)) // sorted keys
maps.Clone(m)                       // shallow copy
maps.Equal(m1, m2)                  // deep compare
```

57.2 — Iterators (Go 1.23+, range-over-func)

```
// Go 1.23 added range-over-func: iterate with custom sequences
import "iter"

// Filter iterator (like LINQ .Where)
func Filter[T any](seq iter.Seq[T], pred func(T) bool) iter.Seq[T] {
    return func(yield func(T) bool) {
        for v := range seq {
            if pred(v) {
                if !yield(v) { return }
            }
        }
    }
}

// Usage:
adults := Filter(slices.Values(users), func(u User) bool {
    return u.Age >= 18
})
```

```
for u := range adults {  
    fmt.Println(u.Name)  
}
```

TIP: Go's iterator pattern (`iter.Seq`) is lazy like C#'s `IEnumerable<T>` with `yield return`. Combined with generics, you can build composable pipelines. However, Go intentionally does not ship a full LINQ-like library — the community provides packages like `samber/lo` for functional helpers.

Lesson 58: Reflection — reflect Package & Performance

Go's reflect package provides runtime type inspection, similar to System.Reflection in C#. It is used by encoding/json, ORMs, and validation libraries. However, reflection in Go is slow and should be avoided in hot paths.

58.1 — C# Reflection vs Go reflect

C# / .NET

```
// C# - System.Reflection
var type = typeof(User);
var props = type.GetProperties();

foreach (var prop in props) {
    var val = prop.GetValue(user);
    Console.WriteLine(
        $"{prop.Name}: {val}");
}
```

Go

```
// Go - reflect package
v := reflect.ValueOf(user)
t := v.Type()

for i := 0; i < t.NumField(); i++ {
    field := t.Field(i)
    val := v.Field(i)
    fmt.Printf("%s: %v\n",
        field.Name, val)
}
```

58.2 — Reading Struct Tags via Reflection

```
type User struct {
    Name string `json:"name" validate:"required"`
    Email string `json:"email" validate:"email"`
}

field, _ := reflect.TypeOf(User{}).FieldByName("Name")
jsonTag := field.Tag.Get("json")      // "name"
valTag := field.Tag.Get("validate")   // "required"
```

58.3 — Performance Implications

Operation	Relative Cost
Direct field access	~1 ns (baseline)
Interface method call	~2-3 ns
reflect.ValueOf()	~50-100 ns
reflect field access	~200-500 ns
reflect method call	~500-1000 ns

WARNING: Reflection is 100-500x slower than direct access. Use generics (Go 1.18+) instead of reflection wherever possible. Libraries like encoding/json/v2 use code generation to avoid reflection overhead in hot paths.

Lesson 59: Observability — Flight Recorder, Metrics, Tracing

Go 1.25 introduced Flight Recorder (runtime/trace), providing always-on tracing with minimal overhead. Combined with pprof, OpenTelemetry, and the expvar package, Go has a comprehensive observability story.

59.1 — Flight Recorder (Go 1.25+)

```
import "runtime/trace"

// Start flight recorder (circular buffer, minimal overhead)
fr := trace.NewFlightRecorder()
fr.Start()

// On error or signal, snapshot the buffer
f, _ := os.Create("trace.out")
fr.WriteTo(f) // writes recent trace data
f.Close()

// Analyze: go tool trace trace.out
// Shows: goroutine scheduling, GC pauses, syscalls,
//        network I/O, blocking events
```

59.2 — OpenTelemetry Integration

```
// Go is a first-class OpenTelemetry citizen
import "go.opentelemetry.io/otel"
import "go.opentelemetry.io/otel/trace"

tracer := otel.Tracer("myservice")

func HandleRequest(ctx context.Context) error {
    ctx, span := tracer.Start(ctx, "HandleRequest")
    defer span.End()

    span.SetAttributes(attribute.String("user.id", userID))

    // Child span propagated via context
    result, err := processOrder(ctx, orderID)
    if err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, err.Error())
        return err
    }
    return nil
}
```

59.3 — expvar (Built-In Metrics)

```
import "expvar"

// Publish metrics at /debug/vars (JSON)
var (
    requests = expvar.NewInt("requests_total")
    errors   = expvar.NewInt("errors_total")
    uptime   = expvar.NewString("uptime")
)

// Increment in handlers
requests.Add(1)

// View: curl http://localhost:8080/debug/vars
// {"requests_total": 42, "errors_total": 3, ...}
```

59.4 — Phase 6 Summary: Advanced Concept Map

C# / .NET Concept	Go Equivalent
volatile / MemoryBarrier	Channels, sync.Mutex, sync/atomic
Gen0/Gen1/Gen2 GC	Concurrent tri-color mark-sweep (no generations)
GCSettings.LatencyMode	GOGC + GOMEMLIMIT env vars
dotTrace / dotMemory	go tool pprof (built-in)
BenchmarkDotNet	go test -bench (built-in)
Tiered compilation	PGO (Profile-Guided Optimization)
unsafe { } block	unsafe package (no compiler flag)
P/Invoke / [DllImport]	cgo (import "C")
gRPC (.NET)	gRPC (google.golang.org/grpc)
ASP.NET Middleware pipeline	func(http.Handler) http.Handler chain
IAHostApplicationLifetime	http.Server.Shutdown + signal.Notify
InnerException chain	fmt.Errorf("%w") + errors.Is/As
LINQ	slices/maps packages + iter (Go 1.23+)
System.Reflection	reflect package (avoid in hot paths)
EventSource / DiagnosticSource	Flight Recorder + expvar + OpenTelemetry

TIP: Go's observability tools are built into the runtime. No NuGet packages needed for profiling, tracing, or metrics. Start every production service with `import _ "net/http/pprof"` and you'll always have profiling available when you need it.